

Limitazioni note e scelte progettuali

Questo documento descrive in modo **onesto e trasparente** i limiti noti dell'applicazione, il *perché* esistono, e come vengono gestiti. È pensato per la revisione tecnica e di sicurezza (Sistemi Informativi).

Il principio guida è: **nessun limite nascosto**. Dove un comportamento non è un difetto risolvibile ma una proprietà della piattaforma, viene dichiarato come tale e "gestito per progettazione", non mascherato.

Indice

1. Modello sincrono di Streamlit (limite principale)
 2. Dipendenza dalla versione di Edge / msedgedriver
 3. Accoppiamento con l'interfaccia di Oracle (XPath)
 4. Una sola operazione alla volta sul server
 5. Riconoscimento del linguaggio naturale (NLP)
 6. Tempi di elaborazione di Oracle
 7. Riepilogo per la revisione
-

1. Modello sincrono di Streamlit (limite principale)

Che cos'è. L'automazione gira **in modo sincrono dentro l'esecuzione dello script Streamlit**. Non c'è un thread in background né una coda di lavori: quando l'utente clicca un pulsante, il browser viene aperto e il codice rimane "bloccato" nell'elaborazione fino a fine operazione.

Perché è un limite. Se la connessione WebSocket tra il browser dell'utente e il server si interrompe e si riconnette **durante** un'operazione lunga (per esempio per un timeout di rete, o perché l'utente ricarica la pagina), Streamlit riesegue lo script da capo e l'automazione in corso viene **abbandonata a metà**.

Perché NON è un bug risolvibile. È una proprietà architetturale di Streamlit, non un errore nel nostro codice. Streamlit non è progettato per processi lunghi in primo piano; non fornisce (in modo nativo e stabile per questo caso d'uso) un meccanismo di job asincroni che sopravviva alle riconessioni.

Come viene gestito (per progettazione): - **Timeout del reverse proxy generoso (30 minuti)**. Configurato da Sistemi Informativi sul proxy che serve `oraclecourseautomator.gruppomagis.it`. Senza questo, il proxy taglierebbe la connessione dopo pochi secondi durante le elaborazioni lunghe. Con 30 minuti, i batch lunghi non vengono interrotti. *Questa è la condizione più critica per il corretto funzionamento in produzione.* - **Avvisi chiari all'utente**. L'interfaccia mostra "Non ricaricare la pagina" durante ogni operazione, e il manuale utente lo ripete. - **Uso una operazione alla volta**. Vedi punto 4.

Impatto residuo. Se, nonostante tutto, una connessione cade a metà di un batch, l'operazione si interrompe. L'utente deve verificare in Oracle cosa è stato effettivamente inserito prima di rilanciare, per evitare duplicati. Il log giornaliero aiuta a ricostruire fin dove era arrivata.

2. Dipendenza dalla versione di Edge / msedgedriver

Che cos'è. L'automazione usa Selenium con Microsoft Edge. Il driver `msedgedriver.exe` deve corrispondere alla versione di Edge installata (le prime tre cifre della versione devono coincidere).

Perché è un limite. Se Edge si aggiorna automaticamente e il driver no, **il browser non si avvia più** e nessuna operazione funziona.

Punto a favore per la sicurezza: il disallineamento **non causa danni silenziosi**. Non c'è rischio di scritture sbagliate su Oracle: semplicemente il browser non parte e l'errore è **immediato e visibile**.

Come viene gestito: - L'aggiornamento automatico di Edge sul server è **disattivato** (vedi `DEPLOYMENT.md`), così la versione resta stabile. - Quando in futuro si deciderà di aggiornare Edge, nella **stessa finestra di manutenzione** va aggiornato anche `msedgedriver.exe` alla versione corrispondente. La procedura è documentata in `DEPLOYMENT.md`. - È una **dipendenza operativa con un titolare da assegnare**: dopo la fine del contratto dello sviluppatore, occorre indicare chi effettua questo aggiornamento congiunto.

3. Accoppiamento con l'interfaccia di Oracle (XPath)

Che cos'è. L'automazione trova i campi e i pulsanti di Oracle tramite XPath (percorsi che identificano gli elementi della pagina). Questi XPath dipendono dalla struttura attuale dell'interfaccia di Oracle HCM.

Perché è un limite. Se un aggiornamento di Oracle cambia la struttura di una pagina, l'XPath corrispondente potrebbe non trovare più l'elemento, e quel passo dell'operazione fallisce.

Come viene gestito (per progettazione): - **Tutti gli XPath sono raccolti in un unico file, `config.py`**, come costanti con un nome. Quando Oracle cambia qualcosa, si corregge lì, senza toccare la logica delle operazioni. - **Molti campi hanno XPath alternativi** provati in ordine, così una piccola variazione di Oracle spesso non rompe nulla. - Un fallimento di questo tipo è **visibile** nel messaggio di riepilogo e nel log giornaliero, che indica a quale passo si è fermato.

Impatto residuo. Dopo un aggiornamento maggiore di Oracle potrebbe rendersi necessario un intervento di manutenzione su `config.py`. È normale per qualsiasi automazione basata su interfaccia web.

4. Una sola operazione alla volta sul server

Che cos'è. Il server esegue **un solo processo** dell'applicazione, che serve tutti i colleghi, e l'automazione è sincrona (vedi punto 1). Due operazioni lanciate contemporaneamente si contenderebbero **lo stesso** browser.

Stato attuale dell'applicazione — dichiarazione onesta: - Esiste una **protezione per sessione** (`automation_in_progress`): impedisce che un singolo utente, con un doppio click o una riconnessione, avvii due browser nella propria sessione. Questa protezione è **attiva**. - Esiste inoltre un **lock a livello di macchina** (`automation_lock.py`), scritto per garantire l'esecuzione esclusiva su tutta la VM, con heartbeat e auto-recupero dei processi orfani. **L'infrastruttura di heartbeat e rilascio è collegata, ma il blocco di ingresso che impedirebbe a un secondo utente di partire (`try_acquire`) NON è attualmente attivato.** Di conseguenza, allo stato attuale, la regola "una operazione alla volta tra utenti diversi" è garantita per **accordo organizzativo** tra i colleghi, non imposta dal software.

Perché questa scelta. Attivare il blocco tra utenti comportava, nelle prime versioni, falsi messaggi di "server occupato" e il rischio di abbandonare automazioni valide. Vista la dimensione del team (poche persone del team formazione) e l'uso coordinato, si è preferito **non imporre** il blocco per non introdurre falsi positivi vicino alla messa in produzione.

Come viene gestito: - Regola organizzativa chiara nel manuale utente: **mettersi d'accordo per non lanciare due operazioni insieme**. - L'infrastruttura del lock resta pronta: se in futuro il team crescerà, attivare l'imposizione richiede un intervento localizzato (chiamare `try_acquire` all'avvio dell'operazione e mostrare la pagina "occupato"). È documentato come evoluzione possibile.

Impatto residuo. Se due colleghi lanciano davvero un'operazione nello stesso momento, i due browser possono interferire. La probabilità è bassa con un team piccolo e coordinato; la conseguenza non è una corruzione di dati Oracle ma il possibile fallimento di una delle due operazioni, visibile nel riepilogo.

5. Riconoscimento del linguaggio naturale (NLP)

Che cos'è. Il metodo "Compilazione con AI" interpreta una frase in italiano ed estrae i dati (nomi, date, orari, numeri persona), usando spaCy (`it_core_news_sm`) con regole (`Matcher`) e un fallback a espressioni regolari.

Perché è un limite. L'estrazione **non è infallibile**: frasi molto inusuali, ambigue o con formattazioni impreviste possono essere interpretate in modo incompleto o errato.

Come viene gestito (per progettazione): - **Anteprima obbligatoria.** Con il metodo AI l'utente vede **sempre** un'anteprima dei dati estratti e può correggerli o annullare **prima** che qualcosa venga scritto in Oracle. L'NLP non agisce mai direttamente. - Il manuale utente invita esplicitamente a controllare l'anteprima.

Impatto residuo. Nessun impatto su Oracle senza conferma dell'utente. Al massimo l'utente deve correggere a mano un campo estratto male, o usare il metodo Form/Excel per quel caso.

6. Tempi di elaborazione di Oracle

Che cos'è. Alcune operazioni (in particolare l'aggiunta di allievi) vengono **accettate** da Oracle ma elaborate con un ritardo interno di Oracle stesso.

Perché è un limite (apparente). Subito dopo l'invio, gli allievi possono non comparire ancora nella lista. Non è un errore

dell'applicazione: è il tempo di elaborazione di Oracle.

Come viene gestito: - Il messaggio finale avvisa l'utente di **attendere qualche minuto** e ricontrollare. - La verifica automatica in tempo reale è **disattivata di default** per non allungare inutilmente le operazioni; il controllo si può fare direttamente in Oracle.

Impatto residuo. Nessuno sui dati; solo la necessità di attendere prima di vedere il risultato.

Riepilogo per la revisione

#	Limite	Natura	Rischio dati	Come gestito
1	Modello sincrono Streamlit	Piattaforma	Nessuna corruzione; possibile interruzione a metà	Timeout proxy 30 min, avvisi, uso singolo
2	Versione Edge/driver	Operativo	Nessun danno silenzioso (fallimento visibile)	Auto-update Edge disattivato, aggiornamento congiunto documentato
3	XPath di Oracle	Manutenzione	Fallimento visibile nel riepilogo	XPath centralizzati in <code>config.py</code> , fallback multipli
4	Una operazione alla volta	Organizzativo	Possibile fallimento di una delle due, non corruzione	Accordo tra colleghi; lock pronto ma non imposto
5	NLP non infallibile	Funzionale	Nessuno (anteprima obbligatoria)	Conferma utente prima di scrivere su Oracle
6	Ritardo elaborazione Oracle	Esterno	Nessuno	Avviso di attendere e ricontrollare

Punto chiave per la sicurezza. Nessuno di questi limiti può causare una **scrittura errata e silenziosa** su Oracle. I fallimenti sono resi visibili nel messaggio di riepilogo e nel log giornaliero (con nome utente e timestamp per ogni operazione, per tracciabilità).